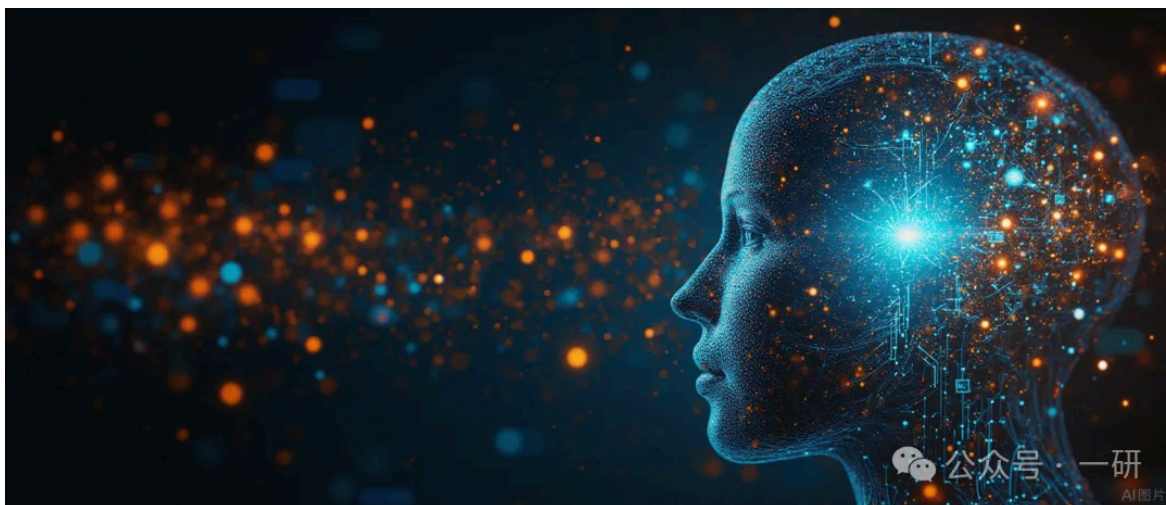


# 【Anthropic Agent智能体圣经】Claude Code 的最佳实践

原创 一研 一研 2026年4月29日 21:06 广东



在📖 2026世界读书日：不荐书，只荐这一份「Agent智能体圣经」清单中提到一份博客清单，本篇为其中的一篇《Claude Code: Best practices for agentic coding》。

本文是一篇译文，为中文读者提供一个了解新世界（Agent智能体系统底层设计逻辑，从而能够更好的理解和使用诸如Claude Code这样的Agent产品）的入口，仍然建议英文好的同学直接查看英文原文。

原文地址：<https://code.claude.com/docs/en/best-practices>

**摘要：**让 **Claude Code** 发挥最大效用的技巧与模式，涵盖从环境配置到并行会话扩展的方方面面。

Claude Code 是一个智能体编程环境（笔者注：编码智能体的鼻祖）。与那种回答问题后就等待的聊天机器人不同，Claude Code 可以读取你的文件、运行命令、进行更改，并自主解决各种问题——你可以在旁观察、做出调整，甚至完全走开。

这改变了你的工作方式：你无需亲自编写代码再让 Claude 审查，而是描述你想要什么，由 Claude 来思考如何构建。

Claude 会探索、规划并实现方案。然而，这种自主性仍然需要一段学习过程。Claude 需要在某些约束条件下工作，你必须理解这些限制。

本指南涵盖的模式已经在 Anthropic 内部团队以及在不同代码库、语言和环境使用 Claude Code 的工程师中证明有效。关于智能体循环的底层工作原理，请参阅 Claude Code 的工作机制<sup>①</sup>。

完整文档索引请访问：<https://code.claude.com/docs/llms.txt>。使用该文件可在进一步探索之前发现所有可用页面。

大多数最佳实践都基于同一个约束条件：**Claude** 的上下文窗口很快就会填满，而性能会随着填满程度而下降。Claude 的上下文窗口容纳你的整个对话，包括每条消息、Claude 读取的每个文

件以及每条命令输出。然而，它可能很快就满了。一次调试会话或代码库探索就可能产生并消耗数万 token。这一点很重要，因为大语言模型的性能会随着上下文被填满而下降。

当上下文窗口接近满时，Claude 可能会开始"遗忘"较早的指令或犯更多错误。**上下文窗口是需要管理的最重要资源**。要了解一个会话在实践中的填满情况，请观看交互式演示②，了解启动时加载了什么以及每次文件读取的成本。使用自定义状态行③持续跟踪上下文使用情况，关于降低 token 使用的策略请参阅降低 token 使用量④。

给 Claude 一个验证自己工作的方式

当 Claude 能够验证自己的工作——比如运行测试、对比截图、校验输出——它的表现会显著提升。没有明确的成功标准，它可能会生成看起来没问题但实际上根本不能用的东西。

你会成为唯一的反馈回路，每个错误都需要你关注。纳入测试、截图或期望输出，让 Claude 能够自我检查。**这是你能做的回报最高的一件事**。UI 更改可以使用 Claude in Chrome 扩展⑤进行验证。它会在你的浏览器中打开新标签页，测试 UI，并反复迭代直到代码正常工作。

你的验证机制也可以是一个测试套件、一个 linter，或者一条检查输出的 Bash 命令。投入精力让你的验证坚如磐石。

| 策略          | 改进前               | 改进后                                                                                              |
|-------------|-------------------|--------------------------------------------------------------------------------------------------|
| 提供验证标准      | "实现一个验证电子邮件地址的函数" | "写一个 validateEmail 函数。测试用例示例：user@example.com 为 true，invalid 为 false，user@.com 为 false。实现后运行测试。" |
| 可视化验证 UI 变更 | "把仪表盘做得更好看"       | "[粘贴截图] 实现这个设计。截取结果截图并与原图对比。列出差异并修复。"                                                            |
| 解决根本原因而非表象  | "构建失败了"           | "构建失败并报此错误：[粘贴错误]。修复它并验证构建成功。解决根本原因，不要只是屏蔽错误。"                                                   |

先探索，然后规划，再动手写代码

让 Claude 直接跳入编码阶段可能会产生解决错误问题的代码。使用规划模式⑥将探索与执行分开。推荐的工作流有四个阶段：

将研究和规划与实施分开，以避免解决错误的问题。

1. 探索：进入规划模式。Claude 读取文件并回答问题，不会进行任何修改。

```
1  claude （规划模式）
2  读取 /src/auth，了解我们如何处理会话和登录。同时查看我们如何管理密钥的环境变量。
```

2. 规划：让 Claude 创建一份详细的实现计划。

```
1  claude （规划模式）
2  我想添加 Google OAuth。哪些文件需要修改？会话流程是怎样的？制定一个计划。
```

按 `Ctrl+G` 在你的文本编辑器中打开该计划，以便在 Claude 继续之前直接编辑。

3. 实现：切换回普通模式，让 Claude 根据其计划编写代码并进行验证。

- 1 `claude`（普通模式）
- 2 按照你的计划实现 OAuth 流程。为回调处理器编写测试，运行测试套件并修复任何失败。

4. 提交：让 Claude 提交代码，并附上描述清晰的提交信息，然后创建一个 PR。

- 1 `claude`（普通模式）
- 2 用描述清晰的提交信息提交，并创建一个 PR

规划模式很有用，但也会增加开销。对于任务范围明确且修改量很小的任务（例如修正错别字、添加日志行或重命名变量），直接让 Claude 处理即可。

当你方法不确定、修改涉及多个文件，或你对要修改的代码不熟悉时，规划是最有用的。如果你能用一句话描述改动内容，就跳过规划。

## 在提示中提供具体上下文

Claude 可以推断意图，但它无法读你的心思。引用特定的文件、提及约束条件、指出示例模式。你的指令越精确，需要的纠正就越少。

模糊的提示在你探索时且可以承受路线修正时是有用的。比如"你会如何改进这个文件？"这样的提示可能会引出你原本没想到要问的问题。

| 策略                           | 改进前                                | 改进后                                                                                                         |
|------------------------------|------------------------------------|-------------------------------------------------------------------------------------------------------------|
| 限定任务范围。指定哪个文件、什么场景以及测试偏好。    | "给 foo.py 添加测试"                    | "为 foo.py 写一个测试，覆盖用户处于登出状态的边缘情况。避免使用 mock。"                                                                 |
| 指向信息来源。引导 Claude 找到能回答问题的来源。 | "为什么 ExecutionFactory 的 API 这么奇怪？" | "查看 ExecutionFactory 的 git 历史，总结其 API 是如何形成的。"                                                              |
| 引用现有模式。向 Claude 指出代码库中的模式。   | "添加一个日历组件"                         | "查看主页上现有组件是如何实现的，以理解其模式。HotDogWidget.php 是一个很好的例子。遵循该模式实现一个新的日历组件，让用户可以选择月份并向前后翻页选择年份。不使用代码库已有库以外的其他库从头构建。" |
| 描述症状。提供症状、可能的位置以及"已修复"的表现。   | "修复登录 bug"                         | "用户报告会话超时后登录失败。检查 src/auth/ 中的认证流程，特别是 token 刷新。写一个能复现该问题的失败测试，然后修复它。"                                      |

## 提供丰富的内容

你可以通过多种方式向 Claude 提供丰富的数据：使用 @ 引用文件、粘贴截图/图片，或直接管道传入数据。

- 使用 @ 引用文件，无需描述代码在哪里。Claude 会在回复前读取该文件。
- 直接粘贴图片。将图片复制/粘贴或拖放到提示中。
- 为文档和 API 参考提供 URL。使用 /permissions 将常用域名加入许可列表。
- 通过运行 `cat error.log | claude` 管道传入数据，将文件内容直接发送给 Claude。
- 让 Claude 获取其所需。告诉 Claude 自己使用 Bash 命令、MCP 工具或读取文件来拉取上下文。

## 配置你的环境

几个设置步骤能让 Claude Code 在所有会话中都显著提升效率。有关扩展功能的完整概述及何时使用，请参阅扩展 Claude Code。

## 编写一份有效的 CLAUDE.md

CLAUDE.md 是一个特殊的文件，Claude 在每次对话开始时都会读取它。其中包含 Bash 命令、代码风格和工作流规则。这为 Claude 提供了它无法仅从代码中推断出的持久上下文信息。运行 `/init` 命令可根据当前项目结构生成一个初始的 CLAUDE.md 文件，后续再逐步完善。`/init` 命令会分析你的代码库，检测构建系统、测试框架和代码模式，为你提供一个可进一步完善的坚实基础。

CLAUDE.md 文件没有强制的格式要求，但务必简洁且易于人类阅读。例如：

```
1 # 代码风格
2 - 使用 ES 模块 (import/export) 语法，而非 CommonJS (require)
3 - 尽可能解构导入 (例如 import { foo } from 'bar')
4
5 # 工作流
6 - 完成一系列代码更改后务必进行类型检查
7 - 优先运行单个测试而非整个测试套件，以获得更好的性能
```

CLAUDE.md 在每次会话都会加载，因此只包含广泛适用的内容。对于仅在某些时候相关的领域知识或工作流，请使用技能Skill代替。Claude 会按需加载它们，而不会让每个对话都臃肿不堪。

务必简洁。对于每一行，问自己："去掉这一行会导致 Claude 犯错吗？"如果不会，就删掉。臃肿的 CLAUDE.md 文件反而会让 Claude 忽略你实际的指令！

| ✅ 包含                 | ❌ 排除                  |
|----------------------|-----------------------|
| Claude 无法猜到的 Bash 命令 | Claude 通过阅读代码就能弄清楚的内容 |
| 与默认风格不同的代码风格规则       | Claude 已知的标准语言惯例      |

| ✅ 包含             | ❌ 排除                |
|------------------|---------------------|
| 测试说明及偏好的测试运行器    | 详细的 API 文档（改为链接到文档） |
| 仓库规范（分支命名、PR 惯例） | 频繁变动的内容             |
| 项目特定的架构决策        | 长篇解释或教程             |
| 开发者环境特性（必需的环境变量） | 对代码库逐文件的描述          |
| 常见陷阱或反直觉的行为      | 不言自明的实践，如"写整洁的代码"   |

CLAUDE.md 文件可以使用以下@path/to/import语法导入其他文件：

```
1 参考 @README.md 了解项目概述，@package.json 了解可用的 npm 命令。
2
3 # 额外说明
4 - Git 工作流：@docs/git-instructions.md
5 - 个人覆盖：@~/ .claude/my-project-instructions.md
```

你可以将 CLAUDE.md 文件放置在以下几个位置：

- 用户主目录（~/ .claude/CLAUDE.md）：应用于所有 Claude 会话
- 项目根目录（./CLAUDE.md）：纳入 git 版本控制以与你的团队共享
- 项目根目录（./CLAUDE.local.md）：个人的项目特定说明；将此文件添加到你的 .gitignore 中，以免与团队共享
- 父目录：适用于 monorepo，其中 root/CLAUDE.md 和 root/foo/CLAUDE.md 会被自动拉取
- 子目录：在处理这些目录中的文件时，Claude 会按需拉取子目录中的 CLAUDE.md 文件

如果尽管有明确规则，Claude 仍持续做你不希望它做的事，那很可能是因为文件太长，规则被淹没了。如果 Claude 问了你一些在 CLAUDE.md 中已有答案的问题，那可能是措辞存在歧义。

请像对待代码一样对待 CLAUDE.md 文件：在出现问题时审查它，定期精简，并通过观察 Claude 的行为是否确实发生变化来测试改动。

你可以通过添加强调（例如，"重要" 或 "你必须"）来调整指令，以改善遵守情况。将 CLAUDE.md 纳入 git，让你的团队也可以贡献内容。随着时间的推移，该文件的价值会不断增长。

配置权限

默认情况下，Claude Code 会请求可能修改系统的操作权限，例如文件写入、Bash 命令、MCP 工具等。这很安全但也很繁琐。当你进行到第十次批准时，你已经不是在真正审查，只是在机械地点击通过。有三种方式可以减少这些打断：

- **自动模式**：使用自动模式Ⓢ让分类器处理审批，一个独立的分类器模型会审查命令，仅阻止看起来有风险的操作：权限升级、未知基础设施或恶意内容驱动的操作。适用于你信任任务的大方向但不想对每一步都点击通过的情况。

- **权限许可列表**：允许你确认为安全的特定工具，如 `npm run lint` 或 `git commit`。
- **沙箱**：启用操作系统级别的隔离，限制文件系统和网络访问，允许 Claude 在定义的边界内更自由地工作。

## 使用 CLI 工具

CLI 工具是与外部服务交互最高效、最节省上下文的方式。如果你使用 GitHub，安装 `gh` CLI。告诉 Claude Code 在与外部服务交互时使用 CLI 工具，如 `gh`、`aws`、`gcloud` 和 `sentry-cli`。

Claude 知道如何使用它们来创建 issue、提交拉取请求（pull request）和阅读评论。即使没有安装 `gh`，Claude 仍然可以使用 GitHub API，但未经认证的请求经常会触发速率限制。Claude 也能有效学习它尚不知道的 CLI 工具。尝试这样的提示："使用 `foo-cli-tool --help` 了解 `foo` 工具，然后用它解决 A、B、C。"

## 连接 MCP 服务器

通过 MCP 服务器<sup>⑩</sup>，你可以让 Claude 从问题跟踪器实现功能、查询数据库、分析监控数据、集成 Figma 设计稿并自动化工作流。运行 `claude mcp add` 连接外部工具，如 Notion、Figma 或你的数据库。

## 设置钩子

钩子<sup>⑪</sup>会在 Claude 工作流的特定节点自动运行脚本。与作为建议的 `CLAUDE.md` 指令不同，钩子是确定性的，能保证动作一定会发生。

Claude 可以为你创建钩子。尝试这样的提示："编写一个钩子，在每次文件编辑后运行 `eslint`" 或 "编写一个钩子，阻止对 `migrations` 文件夹的写入"。直接编辑 `.claude/settings.json` 可手动配置钩子，运行 `/hooks` 可浏览已配置的钩子。

钩子用于那些必须每次零例外执行的操作。

## 创建技能

技能<sup>⑫</sup>可以为 Claude 扩展项目、团队或领域特定的知识。Claude 会在相关时自动应用它们，或者你也可以使用 `/skill-name` 直接调用它们。在 `.claude/skills/` 目录中创建 `SKILL.md` 文件，即可为 Claude 提供领域知识和可复用的工作流。通过添加包含 `SKILL.md` 的目录来创建技能：

```
1 .claude/skills/api-conventions/SKILL.md
```

```
1 ---
2 名称: api-conventions
3 描述: 我们服务的 REST API 设计惯例
4 ---
5 # API 惯例
```



- 6 - URL 路径使用 kebab-case
- 7 - JSON 属性使用 camelCase
- 8 - 列表端点始终包含分页
- 9 - 在 URL 路径中做 API 版本化 (/v1/、/v2/)

技能还可以定义你直接调用的可重复 workflow:

```
1 .claude/skills/fix-issue/SKILL.md
```

```
1 ---
2 名称: fix-issue
3 描述: 修复一个 GitHub issue
4 disable-model-invocation: true
5 ---
6 分析并修复 GitHub issue: $ARGUMENTS。
7 1. 使用 `gh issue view` 获取 issue 详情
8 2. 理解 issue 中描述的问题
9 3. 在代码库中搜索相关文件
10 4. 实现必要的更改以修复问题
11 5. 编写并运行测试以验证修复
12 6. 确保代码通过 lint 和类型检查
13 7. 创建描述清晰的提交信息
14 8. 推送并创建一个 PR
```

运行 `/fix-issue 1234` 即可调用它。对于你希望手动触发、具有副作用的工作流, 使用 `disable-model-invocation: true`。

## 创建自定义子智能体

子智能体<sup>⑫</sup>在它们自己的上下文中运行, 拥有自己的工具集。它们适用于需要读取大量文件或需要专注处理而不使主对话混乱的任务。在 `.claude/agents/` 中定义专门的助手, Claude 可以将任务委派给它们进行隔离处理。告诉 Claude 显式使用子智能体: "使用子智能体审查此代码的安全问题。"

```
1 .claude/agents/security-reviewer.md
```

```
1 ---
2 名称: security-reviewer
3 描述: 审查代码的安全漏洞
4 工具: Read, Grep, Glob, Bash
5 模型: opus
6 ---
7 你是一名高级安全工程师。审查代码中的:
```

- 8 - 注入漏洞（SQL、XSS、命令注入）
- 9 - 认证和授权缺陷
- 10 - 代码中的密钥或凭据
- 11 - 不安全的数据处理
- 12
- 13 提供具体的行引用和修复建议。

## 安装插件

插件⑬可将技能、钩子、子智能体和 MCP 服务器打包成一个来自社区和 Anthropic 的可安装单元。

如果你使用类型化语言，安装一个代码智能插件⑭，即可为 Claude 提供精确的符号导航和编辑后的自动错误检测。

关于如何在技能、子智能体、钩子和 MCP 之间选择，请参阅扩展 Claude Code⑦。

运行 `/plugin` 浏览插件市场。插件无需配置即可添加技能、工具和集成。

## 有效沟通

你与 Claude Code 的沟通方式会显著影响结果的质量。

## 询问代码库问题

当接触一个新代码库时，使用 Claude Code 进行学习和探索。你可以像问另一位工程师一样向 Claude 提问：

- 日志记录是如何工作的？
- 我如何创建一个新的 API 端点？
- `foo.rs` 第 134 行的 `async move { ... }` 是做什么的？
- `CustomerOnboardingFlowImpl` 处理了哪些边缘情况？
- 为什么这段代码在第 333 行调用 `foo()` 而不是 `bar()`？

## 让 Claude 采访你

Claude 会询问你可能尚未考虑到的问题，包括技术实现、UI/UX、边缘情况和权衡。对于较大的功能，先让 Claude 采访你。以一个最小的提示开始，让 Claude 使用 `AskUserQuestion` 工具采访你。一旦规格完整，启动一个全新的会话来执行它。新会话拥有完全聚焦于实现的干净上下文，并且你有了一份书面的规格文档可供参考。

- 1 我想构建[简要描述]。请使用 `AskUserQuestion` 工具详细采访我。询问有关技术实现、UI/UX、

## 管理你的会话

对话是持久且可逆的。充分利用这一点！



## 尽早且频繁地纠正方向

最佳结果源于紧密的反馈循环。虽然 Claude 偶尔能第一次就完美解决问题，但尽早纠正它通常能更快地产生更好的解决方案。

一旦你注意到 Claude 偏离轨道，就及时纠正。

- **Esc**：使用 Esc 键中断 Claude 正在执行的动作。上下文会被保留，你可以重新定向。
- **Esc + Esc 或 /rewind**：按两次 Esc 键或运行 `/rewind` 打开回溯菜单，恢复之前的对话和代码状态，或从选定的消息进行摘要。
- **"撤销那个操作"**：让 Claude 撤销其更改。
- **/clear**：在不相关的任务之间重置上下文。包含无关上下文的长时间会话会降低性能。运行 `/clear` 并以一个融入了你所学内容的、更具体的提示重新开始。

一个干净会话加上更好的提示几乎总是优于一个积累了大量纠正的长时间会话。

## 积极管理上下文

Claude Code 会在接近上下文限制时自动压缩对话历史，保留重要的代码和决策，同时释放空间。在长时间会话中，Claude 的上下文窗口可能会被无关对话、文件内容和命令填满。这会降低性能，有时还会分散 Claude 的注意力。

- 在任务之间频繁使用 `/clear` 彻底重置上下文窗口
- 当自动压缩触发时，Claude 会摘要出最重要的内容，包括代码模式、文件状态和关键决策
- 如需更多控制，运行 `/compact`，例如 `/compact 重点放在 API 更改上`
- 要仅压缩对话的一部分，使用 `Esc + Esc` 或 `/rewind`，选择一个消息检查点，然后选择"从此处摘要"。这会将从该点往后的消息压缩，同时保持较早的上下文完整。
- 在 CLAUDE.md 中自定义压缩行为，例如添加指令："压缩时，始终保留修改文件的完整列表和所有测试命令"，以确保关键上下文在摘要中得以保留
- 对于不需要保留在上下文中的快速问题，使用 `/btw`。答案会显示在可关闭的浮层中，永远不会进入对话历史，因此你可以在不增加上下文的情况下检查细节。

## 使用子智能体进行调查

由于上下文是你的基本约束，子智能体是最强大的可用工具之一。当 Claude 研究代码库时，它会读取大量文件，所有这些都会消耗你的上下文。子智能体在独立的上下文窗口中运行，并返回摘要。

- 通过 "使用子智能体调查 X" 来委派研究任务。它们在独立的上下文中探索，让你的主对话保持干净以用于实现。
- 子智能体探索代码库，读取相关文件，并带着发现结果返回汇报，所有这些都不会弄乱你的主对话。
- 你也可以在 Claude 实现某些东西后使用子智能体进行验证：

- 1 使用子智能体调查我们的身份验证系统如何处理 token 刷新，以及我们是否有任何可以复用的现有
- 2 使用子智能体审查此代码的边缘情况。

## 通过检查点回溯

Claude 会在进行更改前自动创建检查点。Claude 的每个动作都会创建一个检查点。你可以将对话、代码或两者都恢复到之前的任何检查点。

双击 `Escape` 键或运行 `/rewind` 打开回溯菜单。你可以仅恢复对话、仅恢复代码、恢复两者，或从选定的消息进行摘要。详见检查点⑮。

你无需仔细规划每一步，告诉 Claude 尝试一些有风险的方法。如果不起作用，回退并尝试另一种方法。检查点跨会话持久化，因此你可以关闭终端，稍后仍然可以回溯。

检查点仅跟踪 Claude 所做的更改，不跟踪外部进程。这不能替代 git。

## 恢复对话

Claude Code 在本地保存对话。当一项任务跨多个会话时，你无需重新解释上下文。运行 `claude --continue` 从上次中断的地方继续，或使用 `--resume` 从最近的会话中选择。使用 `/rename` 为会话赋予描述性名称，如 "oauth-migration" 或 "debugging-memory-leak"，以便以后查找。将会话视为分支：不同的工作流可以有独立的、持久化的上下文。

```
1 claude --continue # 恢复最近的对话
2 claude --resume   # 从最近的对话中选择
```

## 自动化与规模化

一旦你能高效地使用一个 Claude，就可以通过并行会话、非交互模式和扇出模式倍增你的产出。

到目前为止，一切假设都基于一个人类、一个 Claude 和一个对话。但 Claude Code 可以水平扩展。

本节介绍的技术将向你展示如何完成更多工作。

## 运行非交互模式

使用 `claude -p "你的提示"`，你可以在没有会话的情况下以非交互方式运行 Claude。非交互模式是你将 Claude 集成到 CI 管道、pre-commit 钩子或任何自动化工作流中的方式。输出格式允许你以编程方式解析结果：纯文本、JSON 或流式 JSON。

```
1 # 一次性查询
2 claude -p "解释一下这个项目是做什么的"
3
4 # 脚本的结构化输出
5 claude -p "列出所有 API 端点" --output-format json
6
7 # 实时处理的流式输出
8 claude -p "分析此日志文件" --output-format stream-json
```

## 运行多个 Claude 会话

有三种主要方式可以并行运行会话。并行运行多个 Claude 会话以加快开发速度、运行隔离实验或启动复杂工作流。

- **Claude Code 桌面应用⑩**：可视化地管理多个本地会话。每个会话拥有自己独立的工作树。
- **网页版 Claude Code⑪**：在 Anthropic 的安全云基础设施上的隔离虚拟机中运行。
- **智能体团队⑫**：通过共享任务、消息传递和一名团队领导者，实现多个会话的自动化协调。

你也可以对测试采用类似方法：让一个 Claude 编写测试，另一个编写代码来通过这些测试。

| 会话 A（编写者）                   | 会话 B（审查者）                                                                |
|-----------------------------|--------------------------------------------------------------------------|
| 为我们的 API 端点实现一个速率限制器        | 审查 @src/middleware/rateLimiter.ts 中的速率限制器实现。查找边缘情况、竞态条件以及与我们现有中间件模式的一致性。 |
| 以下是审查反馈：[会话 B 的输出]。请解决这些问题。 |                                                                          |

## 跨文件扇出

对于大型迁移或分析，你可以将工作分发到许多并行的 Claude 调用中：循环遍历任务，对每个任务调用 `claude -p`。使用 `--allowedTools` 为批量操作限定权限范围。你也可以将 Claude 集成到现有的数据处理/处理管道中：

1. **生成任务列表**：让 Claude 列出所有需要迁移的文件（例如，列出所有 2000 个需要迁移的 Python 文件）
2. **编写脚本遍历列表**

```
1 for file in $(cat files.txt); do
2   claude -p "将 $file 从 React 迁移到 Vue。返回 OK 或 FAIL。" \
3     --allowedTools "Edit,Bash(git commit *)"
4 done
```

### 3.先在少量文件上测试，然后大规模运行

根据前 2-3 个文件出现的问题优化你的提示，然后在全套文件上运行。  
`--allowedTools` 标志限制 Claude 可以做什么，这在无人值守运行时很重要。

在开发期间，使用

`claude -p "" --output-format json | your_command --verbose` 进行调试，生产环境中关闭详细输出。

## 使用自动模式自主运行

如需带有后台安全检查的不间断执行，使用自动模式<sup>⑨</sup>。分类器模型会在命令运行前进行审查，阻止权限升级、未知基础设施和恶意内容驱动的操作，同时让常规工作无需提示即可继续。对于使用 `-p` 标志的非交互式运行，如果分类器反复阻止操作，自动模式会中止，因为没有用户可以回退。阈值详见自动模式的回退机制<sup>⑩</sup>。

```
1  claude --permission-mode auto -p"修复所有 lint 错误"
```

## 避免常见的失败模式

以下是常见的错误。尽早识别可以节省时间：

- **大杂烩会话**：你从一项任务开始，然后问 Claude 一件不相关的事，然后又回到第一项任务。上下文中充满了无关信息。**解决方案**：在不相关任务之间使用 `/clear`。
- **反复纠正**：Claude 做错了，你纠正它；它还是错，你再次纠正。上下文中被失败的尝试方法所污染。**解决方案**：在两次失败的纠正之后，`/clear` 并写一个融入你所学内容的、更好的初始提示。
- **过度指定的 CLAUDE.md**：如果你的 CLAUDE.md 太长，Claude 会忽略其中一半内容，因为重要规则被淹没在了噪音中。**解决方案**：无情地精简。如果 Claude 没有该指令就已经做对了，就删除它，或者将其转换为一个钩子。
- **信任-验证差距**：Claude 生成了一个看起来合理的实现，但没有处理边缘情况。**解决方案**：始终提供验证（测试、脚本、截图）。如果你无法验证它，就不要发布它。
- **无限探索**：你要求 Claude "调查" 某件事，但没有限定范围。Claude 读取了数百个文件，填满了上下文。**解决方案**：限定调查范围，或使用子智能体，这样探索就不会消耗你的主上下文。

## 培养你的直觉

本指南中的模式并非一成不变。它们是通常效果良好的起点，但可能并非对每种情况都是最优。

有时你应该让上下文积累，因为你正深入一个复杂问题，历史记录很有价值。有时你应该跳过规划，让 Claude 自行解决，因为任务是探索性的。有时一个模糊的提示恰恰是正确的，因为你想在约束它之前看看 Claude 如何解读问题。

注意观察什么方法有效。当 Claude 产出优秀的成果时，留意你做了什么：提示的结构，你提供的上下文，你所在的模式。当 Claude 遇到困难时，问问自己为什么。是上下文太嘈杂？提示太模糊？还是任务太大无法一次完成？

随着时间的推移，你将培养出任何指南都无法捕捉的直觉。你会知道何时要具体，何时要开放；何时要规划，何时要探索；何时要清除上下文，何时要让它自然积累。

## 相关资源

- Claude Code 的工作机制<sup>⑪</sup>：智能体循环、工具和上下文管理
- 扩展 Claude Code<sup>⑫</sup>：技能、钩子、MCP、子智能体和插件
- 常见工作流<sup>⑬</sup>：调试、测试、PR 等场景的逐步操作指南
- CLAUDE.md<sup>⑭</sup>：存储项目惯例和持久化上下文

## 扩展阅读：

- ①、Claude Code工作机制, <https://code.claude.com/docs/en/how-claude-code-works>
- ②、会话在实践中如何被填满的交互演示, <https://code.claude.com/docs/en/context-window>
- ③、自定义状态行, <https://code.claude.com/docs/en/statusline>
- ④、降低token使用量, <https://code.claude.com/docs/en/costs#reduce-token-usage>
- ⑤、Claude的Chrome扩展, <https://code.claude.com/docs/en/chrome>
- ⑥、规划模式, <https://code.claude.com/docs/en/common-workflows#use-plan-mode-for-safe-code-analysis>
- ⑦、Claude Code扩展, <https://code.claude.com/docs/en/features-overview>
- ⑧、技能Skill, <https://code.claude.com/docs/en/skills>
- ⑨、自动模式, <https://code.claude.com/docs/en/permission-modes#eliminate-prompts-with-auto-mode>
- ⑩、MCP服务器, <https://code.claude.com/docs/en/mcp>
- ⑪、钩子指导, <https://code.claude.com/docs/en/hooks-guide>
- ⑫、子智能体, <https://code.claude.com/docs/en/sub-agents>
- ⑬、插件, <https://code.claude.com/docs/en/plugins>
- ⑭、代码智能插件, <https://code.claude.com/docs/en/discover-plugins#code-intelligence>
- ⑮、检查点, <https://code.claude.com/docs/en/checkpointing>
- ⑯、Claude Code桌面应用, <https://code.claude.com/docs/en/desktop#work-in-parallel-with-sessions>
- ⑰、Claude Code网页版, <https://code.claude.com/docs/en/claude-code-on-the-web>
- ⑱、智能体团队, <https://code.claude.com/docs/en/agent-teams>
- ⑲、常见工作流, <https://code.claude.com/docs/en/common-workflows>
- ⑳、CLAUDE.md, <https://code.claude.com/docs/en/memory>



一元或在看都是莫大的鼓励，一起成长。

 喜欢作者

“ 为者常成，行者常至 ”



AI领域知识 · 目录 ≡

<| 上一篇

【Anthropic Agent智能体圣经】我们如何构建多智能体研究系统

下一篇 >

【Anthropic Agent智能体圣经】为智能体编写有效的工具——借助智能体自身的力量